Object Oriented Programming

Mihai Dascălu

Overview

```
public class Hello {
  public static void main (String[ ] args) {
    System.out.println("Hello world!");
  }
}
```

About me

Full professor @UPB

Education:

- B.Sc., Computer Science, UPB
- M.Sc. UPB & Univ. Nantes
- Ph.D. UPB & Ph.D. Educational Sciences, Univ. Grenoble Alpes
- Senior Fulbright Alumnus, Arizona State University & Post-doc, UPB

Publications:

- Over 200 papers, including top conferences & journals
- **> 3000** citations: https://scholar.google.com/citations?user=3L9yY8UAAAAJ
- US Patent: Three-Dimensional Latent Semantic Analysis

Professional certifications:

- PMP, PMI-RMP, PMI-ACP, CBAP, CISA, CISSP, SSCP, C|EH
- **OCMJD**, **SCJP**, ITIL Foundation, COBIT

Contact:

- mihai.dascalu@upb.ro

THE Team

- **Andrea Duțulescu**
  - MSc student, Research Assistant
- **Gabriel Guțu**
  - PhD, Lecturer, Frontend & Backend Developer
- **Laurentiu Neagu**
  - PhD Student, Research Assistant,
    CEO and Founder – Te Fac Programator
- **Marilena Panaite**
  - PhD Student, Adobe Romania
- Invited speaker: **Victor Rentea** - https://victorrentea.ro
  - Java Champion, Independent Trainer & Coach
  - Spectacular courses on Design Patterns (x3)

Organization

Activity	Points + Bonus
Lab (truncated) 12 x 1 point	10
Quiz of the week (scaled) – each Wednesday @end of course	5
Partial exam	15
Homework (x2)	(10 + 1) x 1.5 x 2 30 + 3
Bonus experiments	3
Final exam	20
Final project	(10 + 1) x 2 20 + 2
Total	**100 + 8**

Labs

- Fist lab – get to know each other – meet & greet with a little bit of introduction to Java
- Scored proportionally to solved exercises
- Truncated to 10 points
- Lab attendance:
  - Remain in assigned half of class (only motivated exceptions will be considered)
  - **Maximum 15 people per lab**
  - Attendance is mandatory
  - Recovery during same week (max 2) – ask your TA

Homework

Discussions @ Friday course

T1: ~1 Nov >> 7 Dec (hard deadline)

- Considers: Data structures & Polymorphism

T2: ~8 Dec >> 11 Jan (hard deadline)

- Considers: Collections, Generics & I/O

Homework (x2)

- (10 points + 1 point bonus) X 1.5 scaling factor
- Submitted on **GitHub**
- 5 points for automated tests
- Code documentation + ReadME/JavaDoc
- **OOP design matters!**
  - After code inspection you can end up with 0 points overall!
- **MOSS check**
- Random sampling for presentation
- Mandatory presentation of all homework if <50% intermediary test
- All deadlines are hard!

Final project

- (10 points + 1 point bonus) X 2 scaling factor
- Assignment given at the beginning of January
- **Deadline**: 2/3 days before the exam
- Submitted on Github
- **MOSS** check & other quality metrics (Unit tests)
- Includes entire course curriculum (design patterns & threads)
- Oral presentation to your TA

Tests

- Course tests
  - Only for practice, similar to the exams
  - Max 5 questions in 8 minutes
- Intermediary test
  - Quiz with **30** questions (1 out of 4) x 1,5 min
  - **2 bonus** questions
  - **Save the date: 18 November @Course**
  - No labs first week
- Exam:
  - Quiz with **60** questions (1 out of 4) x 1,5 min
  - **3 bonus** questions

More info

- No labs in the first week
- No activities in the week Nov 28 – Dec 2
- Please talk to your TA about Dec 22
- Plagiarism will not be tolerated!
- Course pass:
  - Min 30 points from activities throughout the semester
    - If !satisfied => Next year
  - Min 20 points from final exam + final project
    - If !satisfied => Retake exam in September
- No recovery of homework/lab points over summer
- No points are safeguarded in the next academic year

Sole purpose: Learning

Course syllabus

- **Java review** (fundamentals of programming)
  - Data types, arrays, loops, methods
- **Object-oriented programming paradigm**
  - Objects and classes
  - Inheritance and polymorphism
  - Abstract classes and interfaces
  - Internal classes
  - Exception handling
  - String and Text I/O; Binary I/O
  - Generics
  - Collections
- **Design patterns**
- **Multithreading & concurrency**
- **Java profiling and optimization**

Oracle Academy

- Access for everyone to the official curriculum, courses & cumulative final exams:
  - Java Fundamentals
  - Java Foundation
  - Java Programming
- Anonymous accounts were already created
  - Login with your assigned ID & password 'oracleacademy'
  - Change your password after the first login & update personal information
    - academy.oracle.com > Sign in to Student Hub
- Learning resources & quizes are recommended
- Certification vouchers (-25%)

Resources

- Head First: Java, Design Patterns
- Daniel Liang, Introduction to Java Programming (10th edition)
- Bruce Eckel, Thinking in Java (4th Edition)
- OCP Official Certification Books
- Oracle Java Doc
  - https://docs.oracle.com/en/java/javase/index.html

Disclaimer

- Part of the content that follows is inspired from:
  - Daniel Liang's Introduction to Java Programming
  - Sun/Oracle Certified Programmer for Java
  - Stanford CS 108 Object Oriented System Design

Java incentives

Why software engineering?

- You being here ☺
- Creation & creativity
- Immediate feedback
- Constantly changing & learning new things
- Challenge
- Job opportunities
- "Software is the most **malleable of media** ... Software's substrate is the stuff of pure thought."

  (Robert Love - Software Engineer at Google)

Why JAVA?

+ Object oriented
+ Ease of learning / of using (High level)
+ Dynamic & Extensible
+ Efficiency & Robustness
+ Scalability (enterprise level)
+ Platform-independent (“write once, run anywhere”)
+ Portability
+ Profiling

Why Java? (reloaded)

+ Very large programs

+ Multi-threading / tools for parallel coding

+ Breadth of libraries, communities, amount of help
  - Rich API
  - Great collection of open-source libraries
  - Wonderful community support

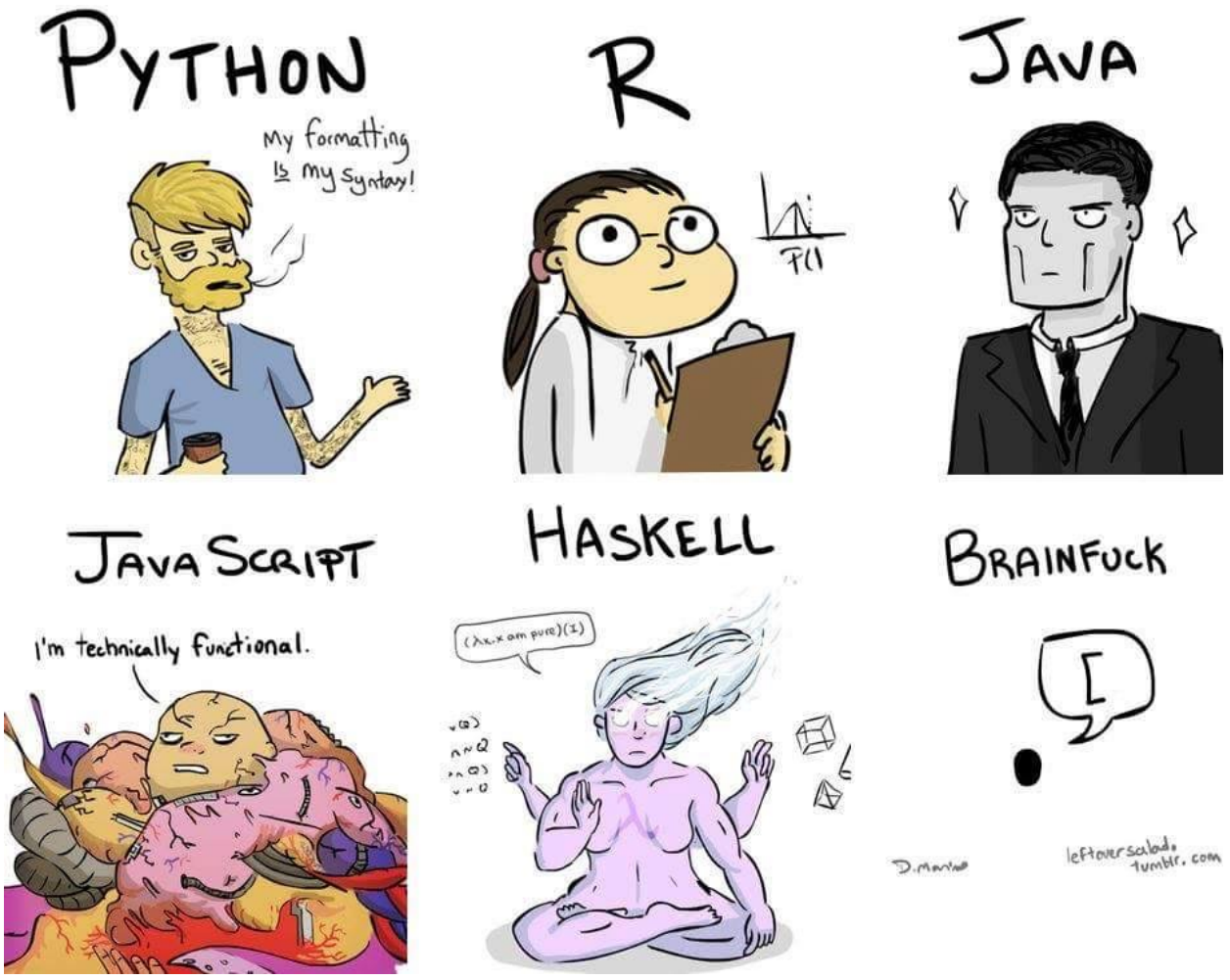
+ „Excellent” documentation support – Javadocs

Java is everywhere

Android, Oracle, IBM, HP...

Even in Minecraft mods

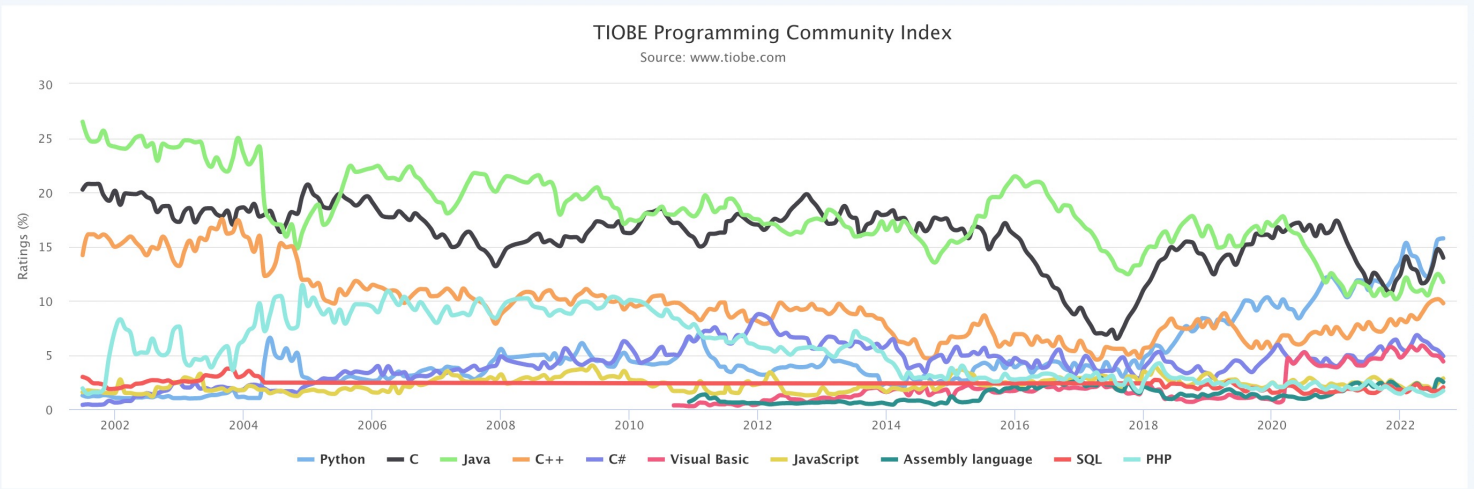
Say What? JAVA!



Java in numbers
https://dev.java/

- 10 Million Java Developers Worldwide
- 56 Billion devices
- Previously stated:
  - 5 million students study Java
  - #1 Choice for Developers
  - #1 Development Platform **(in the cloud)**

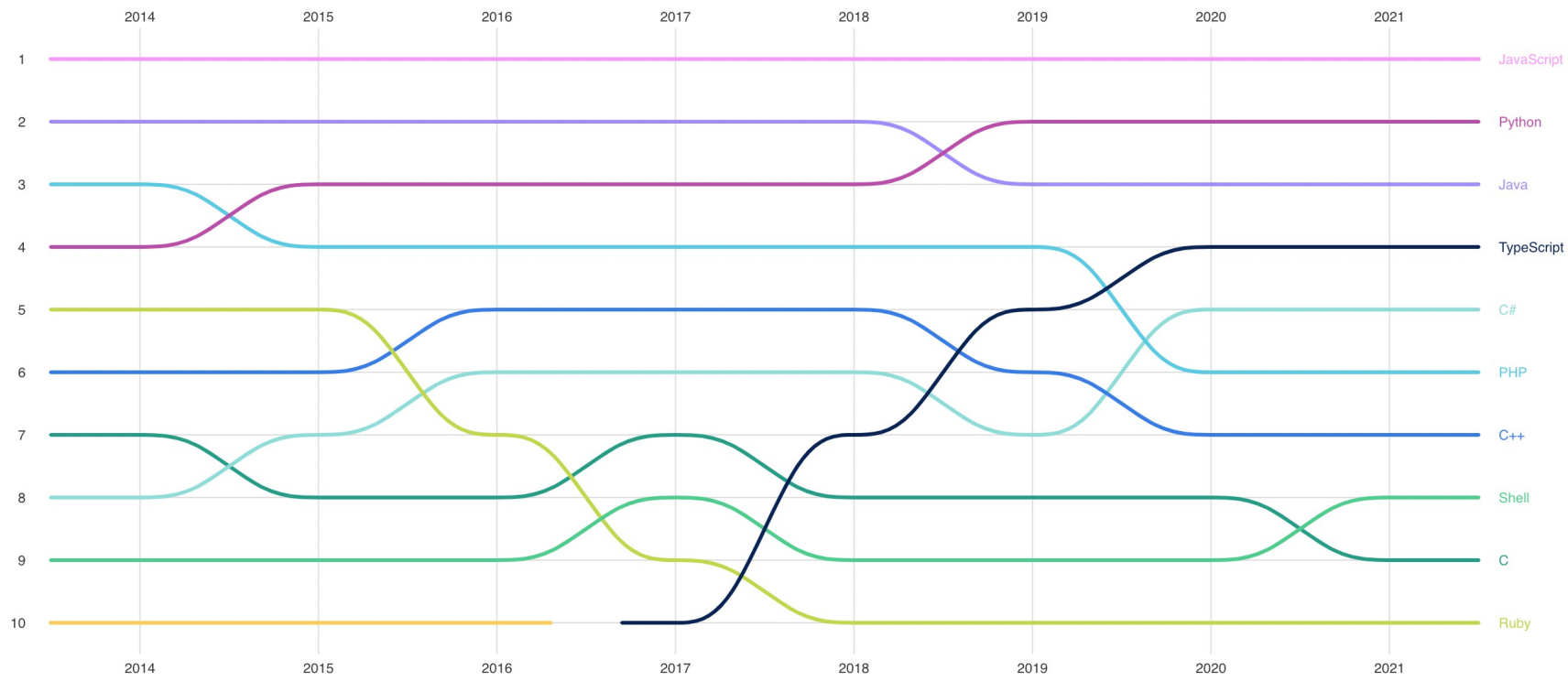
TIOBE Index – Sep 2022



Sep 2022	Sep 2021	Change	Programming Language	Ratings	Change
1	2	▲	Python	15.74%	+4.07%
2	1	▼	C	13.96%	+2.13%
3	3		Java	11.72%	+0.60%
4	4		C++	9.76%	+2.63%
5	5		C#	4.88%	-0.89%
6	6		Visual Basic	4.39%	-0.22%
7	7		JavaScript	2.82%	+0.27%
8	8		Assembly language	2.49%	+0.07%
9	10	▲	SQL	2.01%	+0.21%
10	9	▼	PHP	1.68%	-0.17%

GitHub Popularity

https://octoverse.github.com/#top-languages-over-the-years



What about performance?

- Pointer & direct allocation
- Performance:
  - https://benchmarksgame-team.pages.debian.net/benchmarksgame/fastest/java.html
  - Slower than C/C++, comparable to C# .NET
- Differences to C++
  - https://www.softwaretestinghelp.com/cpp-vs-java/

What's in it for me? Will Java die?

- Java programmers outnumber any other programming language professionals
- Huge availability of Java programmers => large organizations prefer to choose Java for new development than any other programming language
- Difficult to match community, resources and popularity of Java
- Android's underlying platform for development is Java

Java Timeline

https://en.wikipedia.org/wiki/Java_version_history

Roadmap:
https://www.oracle.com/java/technologies/java-se-support-roadmap.html

Release notes:
https://www.oracle.com/java/technologies/javase/19all-relnotes.html
https://openjdk.org/projects/jdk/19/

Version	Release date	End of Free Public Updates[8][9][10][11][12][13][14]	Extended Support Until
JDK Beta	1995	?	?
JDK 1.0	January 1996	?	?
JDK 1.1	February 1997	?	?
J2SE 1.2	December 1998	September 2003	?
J2SE 1.3	May 2000	?	?
J2SE 1.4	February 2002	October 2008	February 2013
Java SE 5	September 2004	November 2009	April 2015
Java SE 6	December 2006	April 2013	December 2018 for Oracle[8] December 2026 for Azul[11]
Java SE 7	July 2011	September 2022 for OpenJDK Maintained by Oracle until May 2015[15], Red Hat until August 2020[16] and Azul until September 2022[17]	July 2022 for Oracle[8] June 2020 for Red Hat[12] December 2027 for Azul[11]
Java SE 8 (LTS)	March 2014	OpenJDK currently maintained by Red Hat[18] March 2022 for Oracle (commercial) December 2030 for Oracle (non-commercial) December 2030 for Azul[11] May 2026 for IBM Semeru[13] At least May 2026 for Eclipse Adoptium[9] At least May 2026 for Amazon Corretto[10]	December 2030 for Oracle[8] November 2026 for Red Hat[12]
Java SE 9	September 2017	March 2018 for OpenJDK	—
Java SE 10	March 2018	September 2018 for OpenJDK	—
Java SE 11 (LTS)	September 2018	OpenJDK currently maintained by Red Hat[19] September 2026 for Azul[11] October 2024 for IBM Semeru[13] At least October 2024 for Eclipse Adoptium[9] At least September 2027 for Amazon Corretto[10] At least October 2024 for Microsoft[20][14]	September 2026 for Oracle[8] September 2026 for Azul[11] October 2024 for Red Hat[12]
Java SE 12	March 2019	September 2019 for OpenJDK	—
Java SE 13	September 2019	OpenJDK currently maintained by Azul[21] March 2023 for Azul[11]	—
Java SE 14	March 2020	September 2020 for OpenJDK	—
Java SE 15	September 2020	OpenJDK currently maintained by Azul[22] March 2023 for Azul[11]	—
Java SE 16	March 2021	September 2021 for OpenJDK	—
Java SE 17 (LTS)	September 2021	OpenJDK currently maintained by SAP[23] September 2029 for Azul[11] October 2027 for IBM Semeru[13] At least September 2027 for Microsoft[14] At least September 2027 for Eclipse Adoptium [9]	September 2029 or later for Oracle[8] September 2029 for Azul[11] October 2027 for Red Hat[12]
Java SE 18	March 2022	September 2022 for OpenJDK and Adoptium	—
Java SE 19	September 2022	March 2023 for OpenJDK	—
Java SE 20	March 2023	September 2023 for OpenJDK	—
Java SE 21 (LTS)	September 2023	September 2028	September 2031 for Oracle[8]

What's new under the hood? A historical view

- **J2SE 5.0 (2004)**
  - Generics, autoboxing, enumerations, varargs, for each loop, Swing updates
- **Java SE 6 (2006)**
  - Dramatic performance improvements in GUI and API, JDBC
- **Java SE 7 (2011)**
  - Concurrency, collections, I/O, internationalization, security & cryptography
- **Java SE 8 (2014)**
  - Closures (lambda expressions) to reduce code
    - Facilitate functional-like coding
  - Diminish the need for anonymous inner classes
- **Java 9 (2017)**
  - Greater modularity
  - Collections & Stream API
  - Jshell: the Java Shell
  - Private methods in interfaces

Oracle JDK vs OpenJDK

- OpenJDK
  - Open-source & free implementation developed by Oracle Corporation, Red Hat, IBM, Apple, OpenJDK & Java Community
  - A feature release every 6 months which is only supported until the next feature release
  - Continuous stream of releases targeted to developers
- Oracle JDK (from 2019):
  - Targets also an enterprise audience which values stability
  - Based on one of the OpenJDK releases, but with long term support (LTS) - planned currently every 3 years
  - Now uses Oracle No-Fee Terms and Conditions License (previously commercial, nooooo!)
- Both are implementations of the same Java specification passed the TCK (Java Technology Certification Kit)
- Almost every existing JDK is derived from OpenJD

Notable Java libraries
(dependent on domain)

- Maven – project object model (POM)
- Log4j 2 – logging
- Junit – unit testing
- Hibernate – persistance
- Frameworks: SpringBoot, Spring MVC, GWT
- APIs: JAX-RS; RESTeasy; Graph-QL-Java
- Akka – parallel processing
- Kafka – stream processing
- Apache Commons
- Elasticsearch, Hadoop, Mahout – Information Retrieval and Big Data
- Apache POI / PDFBox – Java API for Microsoft Documents & PDFs

OOP fundamentals

Object-Oriented Programming fundamentals

- Key concepts:
  - Class versus object (instance of a class)
  - Data fields / attributes / variables & methods (code acting on the data)
- **Encapsulation**
  - Wrap data & methods as a single unit
  - Flexibility & ease of change
- **Abstraction**
  - Reduce the object to its essence
  - Reduce complexity, avoid code duplication and increase reusability

Object-Oriented Programming fundamentals

- **Inheritance** – IS-A relations (Java supports only single inheritance)
  - Pass information down
  - Override & code reuse
- **Subtype polymorphism** versus adhoc (function / operator overloading) versus parametric (generics)
  - Object takes many forms or shapes (any Java object)
  - Compile time polymorphism: method overloading
  - Runtime polymorphism = dynamic method dispatch: method overriding resolved at runtime rather than compile time
  - Allow a single variable to store multiple data types & reduce coupling between different functionalities

Object-Oriented Programming downsides

Versus functional programming: “OOP puts the **Nouns** first and foremost. … It's not as if OOP has suddenly made **verbs** less important … It's a strangely skewed perspective.“ (*Steve Yegge*)

Oversimplification of the real world, *inability* to *model time* properly (*Rich Hickey*)

Limited viewpoint – “almost as much of a hoax as Artificial Intelligence ... It claims that **everything is an object**. Even if it is true it **is not very interesting** - saying that everything is an object **is saying nothing at all** ...“ (*Alexander Stepanov*)

Debatable difference in productivity between OOP and procedural approaches (*Thomas Potok*)

OOP acts as a “**herding mechanism**“ that keeps *mediocre programmers in mediocre organizations from “doing too much damage*“ (*Paul Graham*)